

Senior Challenge 3

Vision AI target detection and follow

Overview

Stack: PX4 SITL + Gazebo (Harmonic) + ROS 2 Humble, with Python executor and vision nodes, and Ultralytics YOLOv8 for detection. This plugs into the `Prompt -> LLM -> validated mission JSON -> deterministic executor -> simulator` pipeline as one new mission step type, `search_and_follow`. Nothing in the vision or control loop calls an LLM see the design decisions section below for why.

Status: verified working end-to-end against live PX4 SITL and Gazebo, twice once on initial integration, and again after a refactor that moved the `NAV_DLL_ACT` arming fix out of a stock PX4 file into a non-clobbering override, with identical results both times. The actual logged output from a verified run is reproduced in the “What to expect on screen” section. The full debugging record lives in `WRITEUP.md` in the repository.

1 Repository layout

vision/ detector.py follow_controller.py snapshot_sender.py config.py	Pure-Python, sim-agnostic. No ROS/PX4 imports. Wraps YOLOv8: frame -> [Detection], class configurable. Deterministic proportional visual-servoing controller. Saves annotated snapshot + operator-notify hook. Loads config/target_config.yaml into typed dataclasses.
executor/ executor.py schema.py actions/ base.py search_and_follow.py takeoff.py, land.py registry.py	Deterministic mission dispatcher (ROS-agnostic). MissionExecutor: runs steps in order via a registry. Per-step required-field checks. ActionHandler interface / ActionResult. The integration point for this challenge. Minimal stub actions. action string -> ActionHandler class mapping.
ros2_nodes/ vehicle_offboard.py camera_subscriber.py mission_executor_node.py	The PX4/ROS2/Gazebo-specific glue. PX4 offboard control wrapper. sensor_msgs/Image -> latest OpenCV frame. rclpy entry point.
sim/ launch/search_and_follow.launch.py worlds/target_actor.sdf	Camera bridge + executor node. Early scaffold, superseded by target_person.
config/target_config.yaml mission/mission_example.json mission/mission_example_car.json	Tunable gains and thresholds. Example mission: takeoff -> follow person -> land. Same mission, following "car" instead.
tests/ test_follow_controller.py	Unit tests of the control law.

<code>test_detector.py</code>	Unit tests of detection filtering/selection.
<code>test_executor.py</code>	Unit tests of the mission dispatch loop.
<code>mock_demo.py</code>	detect -> snapshot -> follow against webcam/video.

2 Configuring the target class

The target class is never hardcoded. It flows from the mission JSON step:

```
{"action": "search_and_follow", "target_class": "person", "return_after_seconds": 30}
```

Changing `target_class` to any COCO class YOLOv8n knows (`car`, `dog`, `bicycle`, ...) requires no other change: `executor/actions/search_and_follow.py` reads it directly off the step and builds the `Detector` with it. `config/target_config.yaml` only holds tuning values (model path, confidence threshold, control gains) plus a `default_target_classes` fallback used solely by `tests/mock_demo.py` when run without a mission JSON.

3 Two ways to run this

- `tests/mock_demo.py` (webcam or video file, no simulator needed): runs the exact same `vision/` code the live ROS 2 node uses, standalone. Use this first, every time, to validate the control loop before touching PX4 or Gazebo.
- **The full PX4 SITL + Gazebo + ROS 2 integration:** the actual deliverable. Everything in `ros2_nodes/` and `sim/` exists only to wire the same `vision/` code up to a real, simulated drone.

3.1 Quickstart no simulator required

```
pip install -r requirements.txt
python3 tests/mock_demo.py source 0 target-class person
```

A window shows the webcam feed. When YOLO first sees a person, the console prints `[mock_demo] ACQUIRED target -> snapshot saved to snapshots/<timestamp>_person.jpg` once, a new annotated JPEG appears in `snapshots/`, and a line is appended to `snapshots/events.jsonl`. Every following frame prints a line such as:

```
state=tracking forward=+0.42 m/s vertical=-0.05 m/s yaw_rate=+0.18 rad/s
```

Walking left, right, closer, or farther from the camera changes `yaw_rate` and `forward` in response the proportional controller reacting to bounding box offset and size in real time. Walking fully out of frame for more than two seconds flips `state` to `lost` and pins all three values to 0.00.

Run the pure-Python unit tests (no camera, model, or ROS required):

```
python3 -m pytest tests/test_follow_controller.py tests/test_detector.py tests/
test_executor.py -v
```

4 Running the full challenge

4.1 Prerequisites

This is exactly what the project was built and verified against, not a general compatibility claim.

- Ubuntu 22.04
- ROS 2 Humble
- PX4-Autopilot, `main` branch, verified against commit `e0137fe7a7` plus this project's own commit on top (see the `NAV_DLL_ACT` fix below)
- Gazebo Harmonic, specifically 8.14.0 (`gz sim version`) PX4's default simulator on current `main`
- Micro XRCE-DDS Agent (eProsima), built from source
- Python 3.10

4.2 Install steps

1. PX4-Autopilot

```
git clone https://github.com/PX4/PX4-Autopilot.git recursive
cd PX4-Autopilot
bash ./Tools/setup/ubuntu.sh
```

This also pulls in the `Tools/simulation/gz` submodule (PX4/PX4-gazebo-models), which now includes `target_person` see the known limitations section for what that is and why it exists.

2. px4_msgs, pinned to tag v1.17.0

```
mkdir -p ~/px4_ros2_ws/src && cd ~/px4_ros2_ws/src
git clone https://github.com/PX4/px4_msgs.git
cd px4_msgs && git checkout v1.17.0
cd ~/px4_ros2_ws && source /opt/ros/humble/setup.bash
colcon build packages-select px4_msgs
```

This pin is load-bearing, not cosmetic. The `px4_msgs` message definitions must match the wire format the PX4 firmware actually sends, byte for byte. If `px4_msgs` is on a different commit than the firmware, the symptoms are silent: ROS 2 discovery matches fine (same topic name, same type name, same QoS), so `ros2 topic info -v` shows a connected publisher and subscriber, but every sample fails a DDS type-consistency check and is silently dropped. `ros2 topic echo` shows zero messages with no error anywhere. This was hit for real during integration: `px4_msgs` was on `main` (`VehicleStatus.MESSAGE_VERSION=4`) against a firmware build reporting `MESSAGE_VERSION=1`. Pinning `px4_msgs` to the tag matching the firmware's release line is the fix; `v1.17.0` is what this project's firmware checkout matched at the time of verification.

3. ros_gz_image, built from source, not the apt package

```
cd ~/px4_ros2_ws/src
git clone https://github.com/gazebosim/ros_gz.git -b humble
cd ~/px4_ros2_ws && GZ_VERSION=harmonic colcon build \
  packages-select ros_gz ros_gz_image ros_gz_bridge ros_gz_sim ros_gz_interfaces
```

The apt package `ros-humble-ros-gz-image` is linked against `libignition-transport11/libignition-msgs8` the older Ignition Fortress-era Gazebo generation. This project's PX4 checkout runs Gazebo Sim 8 (Harmonic), which speaks `gz-transport13/ gz-msgs10`: a wire-incompatible generation. The apt-built bridge does not crash and does not error clearly; it silently logs `Unknown message type [8]/[9]` instead of producing any `sensor_msgs/Image` on the ROS side. This was confirmed directly via `ldd $(ros2 pkg prefix ros_gz_image)/lib/ros_gz_image/image_bridge` the apt binary has no `gz-transport13` in its link list at all. Building from source with `GZ_VERSION=harmonic` set is the fix. This affected both `ros_gz_bridge`'s `parameter_bridge` and `ros_gz_image`'s `image_bridge`; this project uses `image_bridge` specifically because `parameter_bridge` additionally fails on `sensor_msgs/msg/Image` with an unrelated `get_factory()` lookup error even after the generation mismatch is fixed. That was not root-caused further `image_bridge` sidesteps it and was confirmed working end-to-end at roughly 2730 Hz.

4. Micro-XRCE-DDS-Agent

```
git clone https://github.com/eProxima/Micro-XRCE-DDS-Agent.git
cd Micro-XRCE-DDS-Agent && mkdir build && cd build
cmake .. && make -j$(nproc) && sudo make install && sudo ldconfig
```

5. Python dependencies the numpy<2 constraint

```
cd <this repo>
pip install -r requirements.txt
```

`requirements.txt` pins both `numpy<2` and `opencv-python>=4.9.0,<5`. `ros-humble-cv-bridge` (apt) is a compiled extension built against Ubuntu 22.04's stock NumPy 1.x C API. NumPy 2.0 broke C API ABI compatibility, so an unpinned `ultralytics` or `opencv-python` install resolves to NumPy 2.x today (`opencv-python>=5.0` hard-requires `numpy>=2` as a dependency), and `cv_bridge` then dies on import with `AttributeError: _ARRAY_API not found`. This was fixed by pinning both together (`numpy==1.26.4`, `opencv-python==4.9.0.80` resolved) and verified with a real `cv_bridge.cv2_to_imgmsg / imgmsg_to_cv2` round trip showing the array came back byte identical.

6. The NAV_DLL_ACT=0 airframe override

PX4's `NAV_DLL_ACT` parameter defaults to 2, which requires an active ground control station data-link connection before PX4 will allow arming (see `src/modules/commander/HealthAndArmingChecks/checks/rcAn`). This project arms and flies entirely through the ROS 2 offboard link, with no ground control station ever attached, so that check can never pass at its default value every arm attempt returns `VEHICLE_CMD_RESULT_TEMPORARILY_REJECTED` with no further diagnostic beyond `commander: Arming denied: Resolve system health failures first`.

The fix ships as part of this project's PX4-Autopilot checkout, as two files:

```
PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/4001_gz_x500.post
PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/4010_gz_x500_mono_cam.post
```

Each contains one line: `param set-default NAV_DLL_ACT 0`. These are not edits to the stock `4001_gz_x500/ 4010_gz_x500_mono_cam` airframe files they use PX4's own `.post` autostart-override mechanism (`rcS: [ -e "$autostart_file".post ] && . "$autostart_file".post`), so the fix

survives a `git pull` of PX4-Autopilot untouched. Both files are needed because 4010 sources 4001 as a plain shell include rather than through `SYS_AUTOSTART`, so `rcS`'s `autostart-post` lookup only ever checks the top-level selected airframe's own `.post` file a future `x500_*` camera variant would need its own copy too.

If these files are missing for some reason, they can be reapplied manually:

```
echo 'param set-default NAV_DLL_ACT 0' > \
  PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/4001_gz_x500.post
echo 'param set-default NAV_DLL_ACT 0' > \
  PX4-Autopilot/ROMFS/px4fmu_common/init.d-posix/airframes/4010_gz_x500_mono_cam.post
```

Both filenames also then need adding next to their non-`.post` counterparts in `ROMFS/px4fmu_common/init.d-posix`. a `.post` file sitting on disk without a `CMakeLists.txt` entry silently does not get bundled into the built firmware.

4.3 Exact run sequence three terminals

```
# Terminal 1 PX4 SITL + Gazebo
cd <PX4-Autopilot>
make px4_sitl gz_x500_mono_cam

# Terminal 2 PX4 <-> ROS 2 bridge
MicroXRCEAgent udp4 -p 8888

# Terminal 3 this challenge's deliverable
cd <this repo>
source /opt/ros/humble/setup.bash
source ~/px4_ros2_ws/install/setup.bash # must come after humble's setup.bash
                                         # this overlays the correct Harmonic-built
                                         # ros_gz_image and the v1.17.0-pinned
                                         # px4_msgs ahead of any apt-installed
version
ros2 launch sim/launch/search_and_follow.launch.py \
  mission:=mission/mission_example.json \
  vision_config:=config/target_config.yaml
```

Spawn the detectable target into the running world (needed once per Gazebo session it is lost on a Gazebo restart, not a PX4-only restart):

```
gz service -s /world/default/create \
  retype gz.msgs.EntityFactory retype gz.msgs.Boolean \
  timeout 2000 \
  req 'sdf_filename: "<PX4-Autopilot>/Tools/simulation/gz/models/target_person/model.sdf"'
```

4.4 What to expect on screen

This is the actual logged output from a verified run; paths and timestamps will differ, everything else should match.

```
[INFO] [launch]: All log files can be found below /home/<you>/.ros/log/...
```

```
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [image_bridge-1]: process started with pid [...]
[INFO] [python3-2]: process started with pid [...]
[python3-2] [INFO] [...] [mission_executor]: step 0: success=True message=reached
takeoff altitude 5.0m
[python3-2] [INFO] [...] [mission_executor]: step 1: success=True message=
search_and_follow completed after 30.0s (final state: tracking)
[python3-2] [INFO] [...] [mission_executor]: step 2: success=True message=landed
[python3-2] [operator-notify:noop] target acquired -> snapshots/<timestamp>_person.jpg
(no operator_endpoint configured, see snapshot_sender._notify_operator)
[INFO] [python3-2]: process has finished cleanly [pid ...]
```

Concretely: the drone arms and climbs to 5 m (takeoff), then `search_and_follow` runs while it searches, and as soon as YOLO detects `target_class` in the bridged camera feed a snapshot lands in `snapshots/` (filename pattern `<YYYYMMDD-HHMMSS>_<class_name>.jpg`, for example `20260714-191536_person.jpg`), and the drone begins adjusting yaw, altitude, and forward speed to center the target and hold standoff distance. This is visible as real, changing velocity in `/fmu/in/trajectory_setpoint` correlated with real, changing `/fmu/out/vehicle_local_position_v1`, and visible as the vehicle turning and approaching in the Gazebo GUI. `final state: tracking` in the log line above means the target was still actively being tracked, not lost, when `return_after_seconds` elapsed. After that it hovers and the executor moves on to `land`.

5 Known limitations

5.1 No internet access, so the target is a locally-built textured box

The development machine has no internet access, so the detectable target is a locally built textured box rather than a Fuel-hosted person mesh. The original scaffold (`sim/worlds/target_actor.sdf`) documented three options for a YOLO-detectable target; option 2, a Fuel-hosted humanoid mesh, needs an internet fetch this machine cannot do, so option 3, texturing a box with a real photo, was built instead.

1. **The photo:** matplotlib's own bundled sample data (`grace_hopper.jpg`, vendored with the `matplotlib` pip package, so no external network fetch is required) was used because it is a real photograph of a person already present on disk. See `SOURCES.md` for the full provenance and license note, including a flagged-but-not-independently-verified point about the photo's public domain status.
2. **Verified detectable before it was ever wired into Gazebo:** before committing to this photo, it was run directly through this project's own `vision.detector.Detector`, the identical class the live mission uses, not a stand-in. Result: `person` classified at 0.91 confidence, a single clean detection, no ambiguity. Only after that check did it get wrapped into a Gazebo model.
3. **The model:** `PX4-Autopilot/Tools/simulation/gz/models/target_person/`, a thin box (0.05 m by 1.536 m by 1.8 m, life-size and aspect-ratio-matched to the source photo) textured on every face using the same `<material><pbr><metal><albedo_map>` pattern the stock `arucotag` model already uses in this PX4-gazebo-models checkout, so no new Gazebo material system knowledge was needed.
4. **Verified detectable again, live, in Gazebo:** a frame grabbed directly from the live

/camera ROS topic, not a saved test image, was run through the same `Detector`: `person` at 0.91 confidence. During an actual `search_and_follow` mission run, one of the saved snapshots (`snapshots/20260714-175121_person.jpg`) visibly shows the drone's own propeller blades in the frame corners, confirming this is genuinely the live onboard camera feed and not a static or replayed image.

5.2 Other known limitations

- Only tested with a single target instance in frame. `vision.detector.pick_target` explicitly picks the single largest bounding-box match when multiple detections of the same class exist; multi-target tracking and re-identification are out of scope.
- No occlusion recovery beyond the `lost_target_timeout_s` grace period (default 2.0 s) a longer occlusion drops the controller into `LOST` and it hovers rather than searching a pattern to reacquire.
- SITL only this was never run against real PX4 hardware or a real camera/gimbal. See `WRITEUP.md`, "How this would scale," for what would need to change.

6 Design decisions

Why proportional-only, three independent loops, and not a coupled MPC or LQR controller?

The brief asks for simple visual servoing with proportional control on bounding box offset and size. A decoupled P-per-axis controller is the textbook image-based visual servoing baseline: it is trivial to reason about and tune (three gains, each with an obvious physical meaning), and it is easy to bound and clamp for safety. The full derivation and the reasoning behind each axis is in the `vision/follow_controller.py` module docstring.

Why bounding-box height, not width or area, as the distance proxy?

Apparent height is a more stable estimator of distance for people or vehicles than width, since width varies substantially with viewing angle a person or car seen face-on versus side-on has very different width but comparable height. It is also a plain function of an already-available field, with no calibration step beyond what YOLO already provides.

Why EMA smoothing and deadbands, and not a PID controller?

No derivative or integral term was added, deliberately. With a discrete and sometimes noisy detector signal, a naive derivative term would mostly amplify pixel jitter, and an integral term risks windup while the target is lost. EMA smoothing damps jitter ahead of the proportional term instead, and deadbands absorb the residual noise the EMA does not fully remove. This is a conscious simplicity-over-completeness trade for a challenge scoped to simple visual servoing.

Why is vision/ completely free of ROS and PX4 imports?

So the detection and control logic can be unit tested and demoed (`tests/mock_demo.py`) without a simulator, and so it is not tied to PX4 the same module would plug into a ground robot's Nav2 stack by writing a different `ros2_nodes/vehicle_offboard.py` equivalent, without touching `vision/` at all.

Why does `executor/executor.py` also avoid ROS and PX4 imports?

It is the literal deterministic-executor boundary described in the system brief it should remain reusable even if the vehicle stack underneath changes. All stack-specific behaviour is injected through `ExecutorContext` and lives in `ros2_nodes/` and in individual `ActionHandler` subclasses.

Why is the PX4 offboard heartbeat on its own timer, decoupled from the vision loop?

PX4 requires `OffboardControlMode` at more than 2 Hz continuously, or it will auto-exit offboard mode. If that publish were driven by the same loop iteration as YOLO inference, one slow frame a large image, CPU contention, the model warming up could stall the heartbeat and PX4 would kick itself out of offboard mid-follow. Decoupling it onto its own ROS timer means the vehicle interface's safety-critical heartbeat can never be starved by the vision pipeline's variable inference time.

No LLM anywhere in this loop

`follow_controller.py` makes zero network calls and imports no LLM client of any kind it is arithmetic on floating point numbers, driven by `dt` and a `Detection`. This satisfies the requirement for deterministic closed-loop control code literally, and it is also good practice: control decisions running at 10 Hz should not be gated on network latency and a nondeterministic model.

7 Verification history

This module was originally authored in a sandboxed environment with no Python interpreter, no network access, and no ROS, Gazebo, or GPU installed, so nothing could be executed at that time the code was written carefully against documented APIs and reviewed by hand only.

It has since been verified end-to-end against live PX4 SITL and Gazebo, twice, on a real machine with shell access.

1. **First integration pass:** found and fixed the `px4_msgs` version mismatch, the `NAV_DLL_ACT` arming block, and the `ros_gz_image` Gazebo-generation mismatch (see `WRITEUP.md` for the full account). Confirmed `takeoff` -> `search_and_follow` -> `land`, all `success=True`, with real position telemetry correlated to real velocity setpoints.
2. **Second pass**, after refactoring the `NAV_DLL_ACT` fix out of the stock airframe files and into the non-clobbering `.post` override: a full restart from scratch fresh PX4 process, fresh Gazebo world, target respawned re-running the identical mission through the identical launch command produced identical results.

See `WRITEUP.md` for the full narrative and `SOURCES.md` for exactly what was, and was not, independently verified versus assumed.